DEEP LEARNING (ANN)

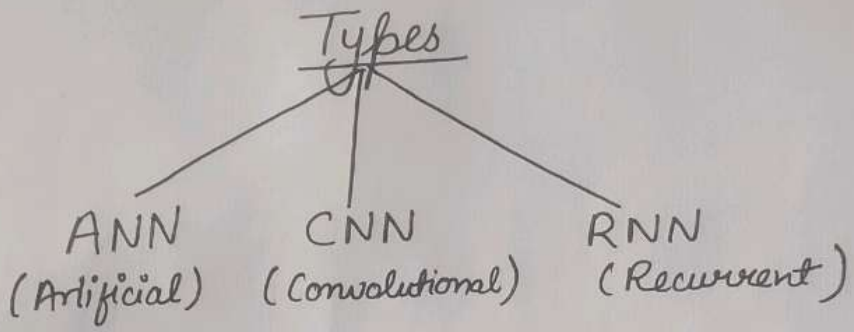
NOTES

Introduction to Neural Network

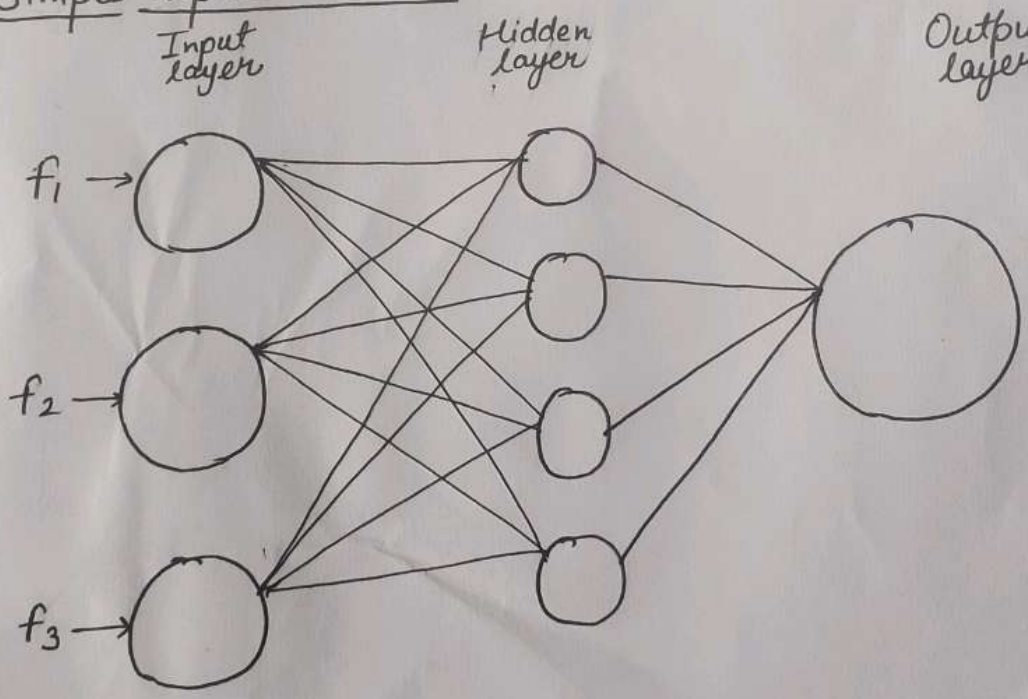
Simplest Neural network :- Perceptron
(It was not much efficient in learning)

In 1980s,

Jeffry Hinton invented Back Propagation



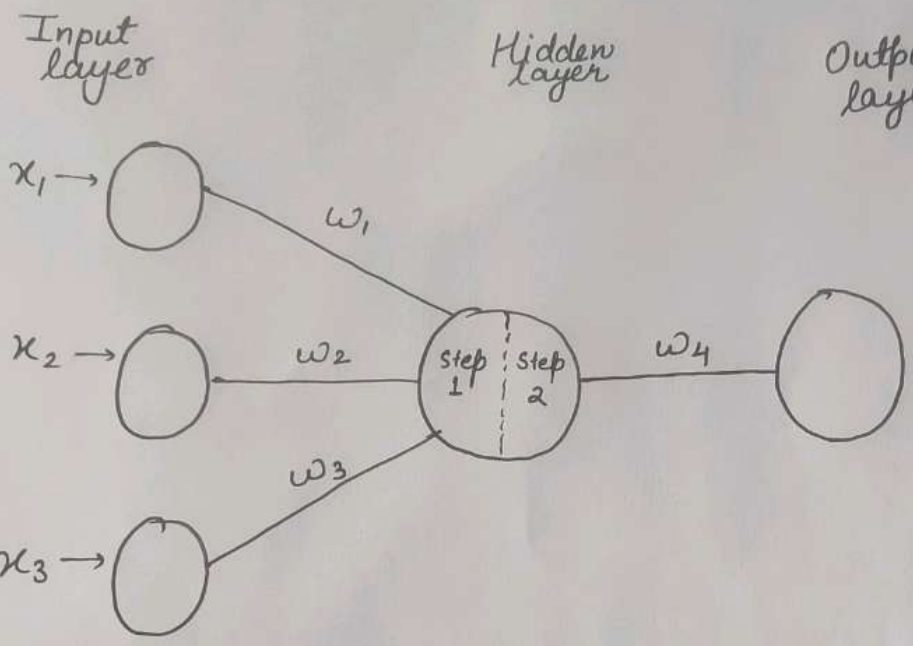
- Simple Representation



How Neural Network Works

This is how forward propagation works.

Perceptron



Here, x_1, x_2, x_3 } Input features

w_1, w_2, w_3, w_4 } weights

* In Hidden layer

Two step take place

Step 1:

$$y = w_1x_1 + w_2x_2 + w_3x_3 + \text{bias}$$

∴ Bias will be covered later.

Step 2:

$$Z = Act(y)$$

∴ y is sent under a activation function like sigmoid, ReLu, etc.

In Output Layer

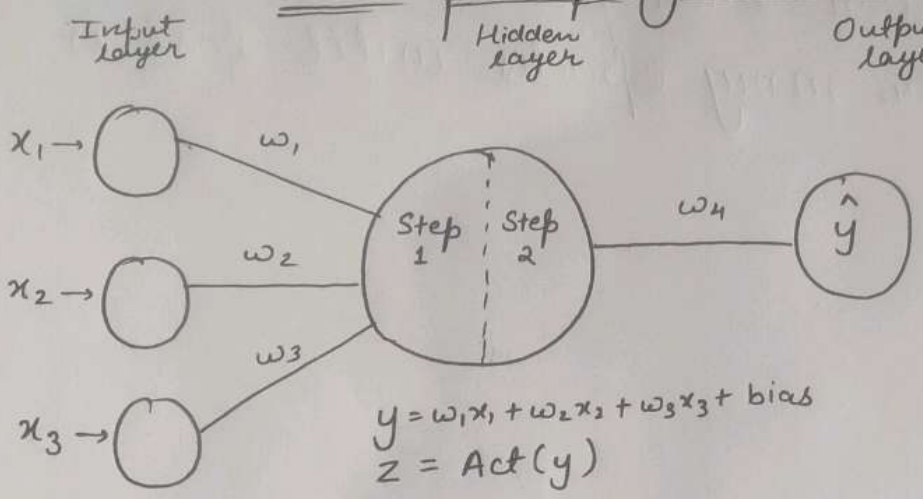
$$k = w_4 \times Z$$

Then,

result = Act(k)

∴ Result will be our desired output.

Backpropagation



y → Actual
$\hat{y}$ → Predicted

Suppose $\hat{y} = 0$
$y = 1$

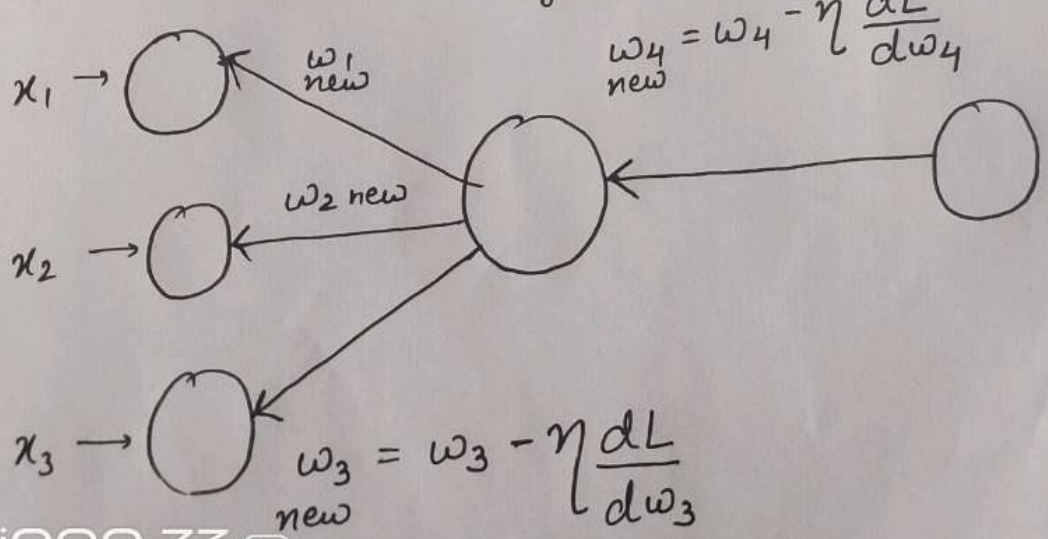
$y = \omega_1 x_1 + \omega_2 x_2 + \omega_3 x_3 + \text{bias}$
$z = Act(y)$

$$Loss = (y - \hat{y})^2$$
$$= (1 - 0)^2$$
$$= 1$$

∴ We have to reduce the loss as much as possible.
for this. we use.
Optimizers (minimize loss)

Now we will do Backpropagation to update weights in such a way that loss is minimized.

(0.01)
∴ η :- learning rate.
dL : derivative of loss.

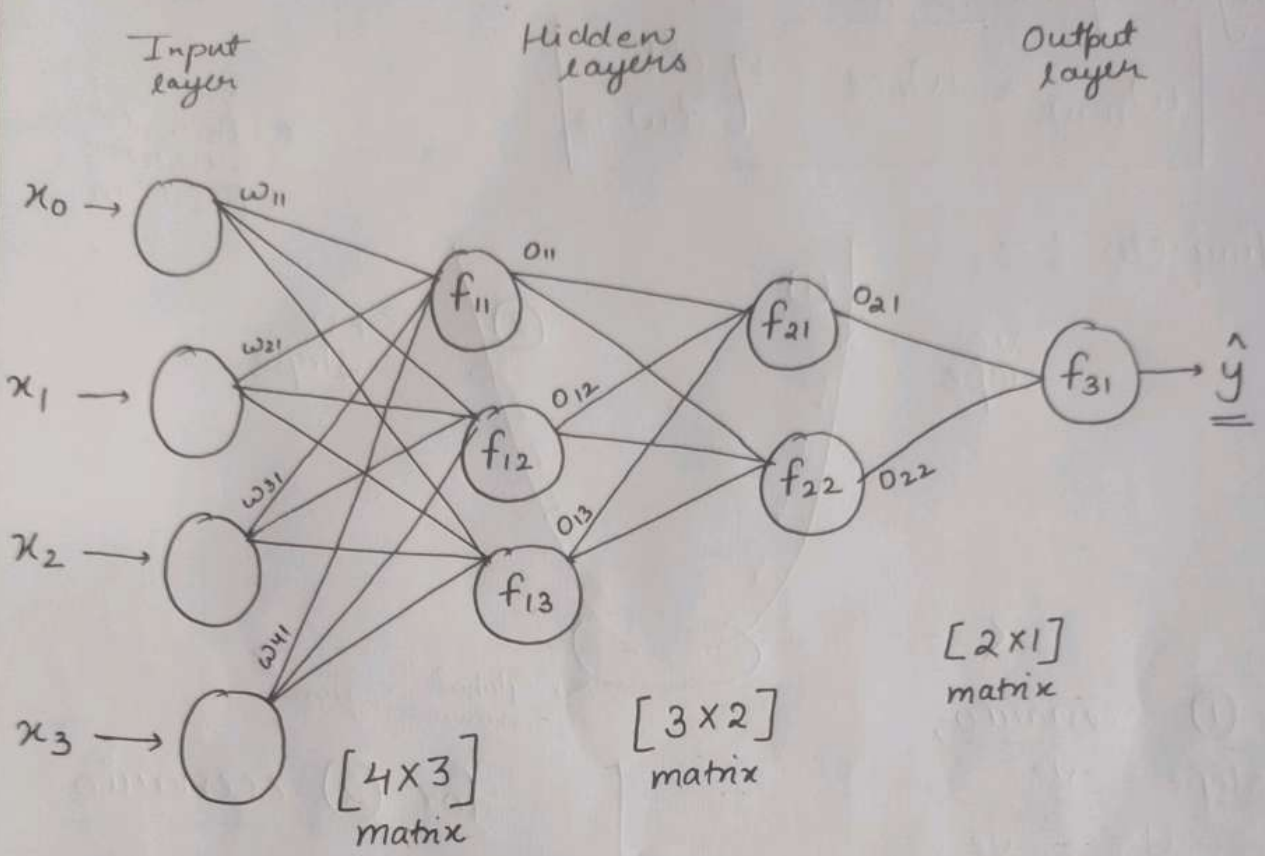


$$\omega_4^{new} = \omega_4 - \eta \frac{dL}{d\omega_4}$$

$$\omega_3^{new} = \omega_3 - \eta \frac{dL}{d\omega_3}$$

When all weights get updated,
Forward Propagation takes place.
This is done for many epochs until loss is minimized.

Train Multi-Layer Neural Network



Here,

$w \Rightarrow$ weights.

$O \rightarrow$ output of respective neuron

Suppose forward propagation is done and we get $\hat{y}$

Now we compare it with actual value using loss function

$$\text{Loss fn} = (y - \hat{y})^2$$

Main aim :- To reduce loss fn

for this we use optimizers.

Here gradient descent is used.

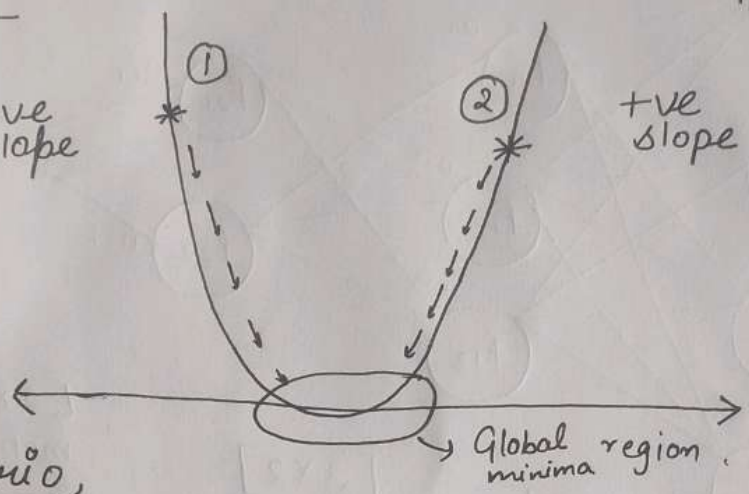
Now Back Propagation takes place.

Weight updation formula,

$$\omega_{new} = \omega_{old} - \eta \frac{dL}{d\omega_{old}}$$

* → Initial weight position.

Optimizer :-



for ① scenario,

slope = -ve.

$\therefore \frac{dL}{d\omega_{old}} = -ve.$

So,

$\omega_{new} = \omega_{old} - \eta\left(\frac{-dL}{d\omega_{old}}\right)$

$\therefore - - = +$

$\omega_{new} = \omega_{old} + \eta \frac{dL}{d\omega_{old}}$

It means we are adding some value to old weight.

for ② scenario.

slope = +ve

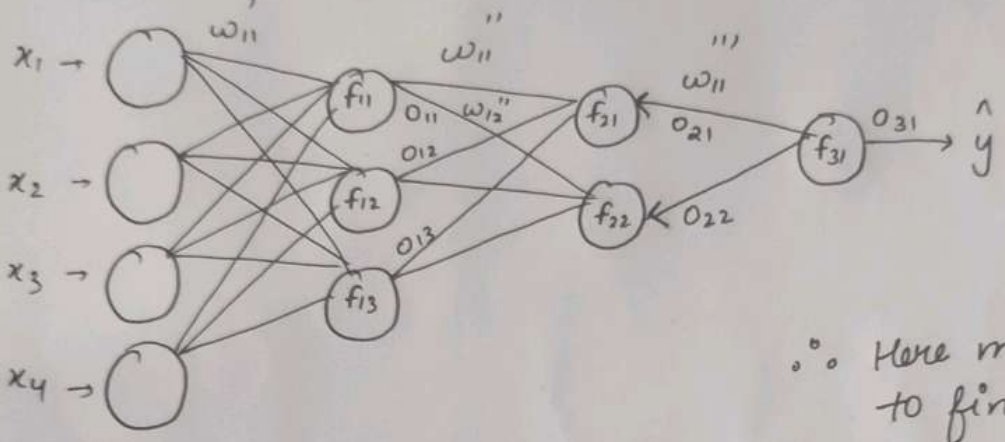
$\therefore \frac{dL}{d\omega_{old}} = +ve.$

It means we are subtracting some value from old weight.

In above both cases weights get close to global minima, it means reducing loss.

Speed of weight updation is determined by η (learning rate). It is generally 0.01.

- **Chain Rule of Differentiation**



∴ Here main point is to find $\frac{dL}{d\omega}$
It indicates slope.

Now,

(i) $\omega_{11}''' = \omega_{11}'''_{old} - \eta \frac{dL}{d\omega_{11}'''_{old}}$

Here, $\frac{dL}{\omega_{11}'''} = \frac{dL}{dO_{31}} \cdot \frac{dO_{31}}{d\omega_{11}'''}$

Here, fn impacted is O_{31}.

(ii) $\omega_{11}'' = \omega_{11}''_{old} - \eta \frac{dL}{d\omega_{11}''_{old}}$.

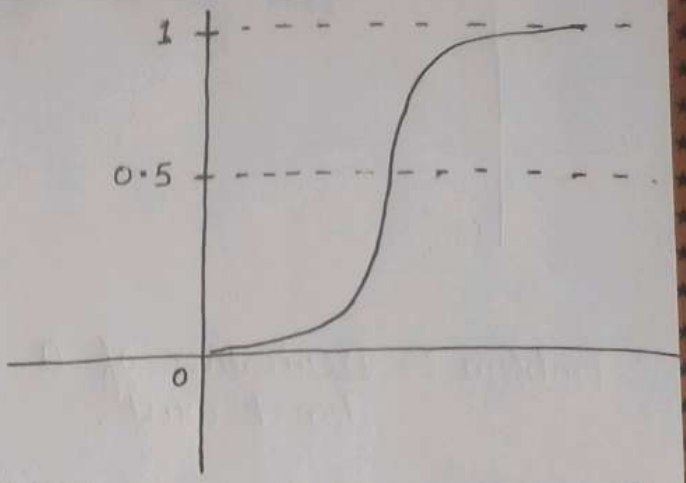
Here,

$$\frac{dL}{d\omega_{11}''} = \left[\frac{dL}{dO_{31}} \cdot \frac{dO_{31}}{dO_{21}} \cdot \frac{dO_{21}}{d\omega_{11}''}\right] + \left[\frac{dL}{dO_{31}} \cdot \frac{dO_{31}}{dO_{22}} \cdot \frac{dO_{22}}{d\omega_{12}''}\right]$$

∴ There are two paths.
So this is done.

Activation Functions - 1

① Sigmoid AF



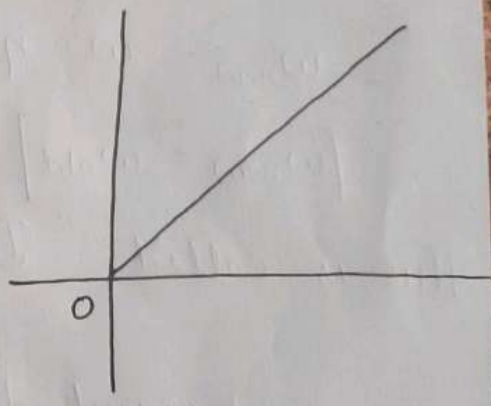
Formula :- $\frac{1}{1+e^{-y}}$

Range :- 0 to 1

Threshold :- 0.5

∴ If value less than 0.5, consider it 0. (neuron deactivated)
value more than 0.5, consider it 1. (neuron activated)

② ReLu AF (Rectified Linear unit)

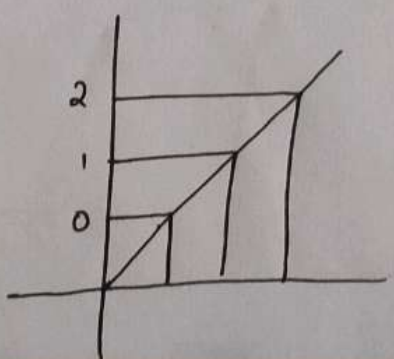


Formula :- $\max(y, 0)$

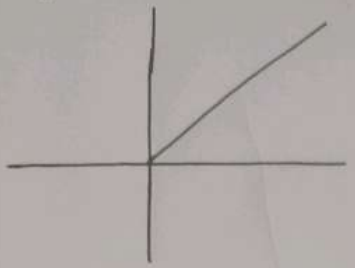
If y is -ve, value will be 0.
y is +ve, value will be 'y'

It means.
If y is 1, value = 1
y is 2, value = 2
and so on

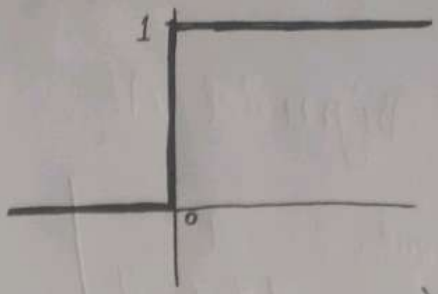
eg.



Graph of ReLu



Graph of derivative of ReLu



$\therefore$ Problem :- Derivative of 0 don't exist. (either 0 or 1)

eg. Suppose we have 3 weights and one is a negative weight. On using ReLu, during backpropagation

Chain Rule = $0 \times 1 \times 2$

$= 0$

So, $\frac{dL}{dw} = 0$.

$w_{new} = w - \eta \frac{dL}{dw}$

$\therefore \boxed{w_{new} = w_{old}}$

This is called "Dying ReLu Problem"

$\Rightarrow$ To prevent this we use leaky ReLu AF

Activation Function - 2

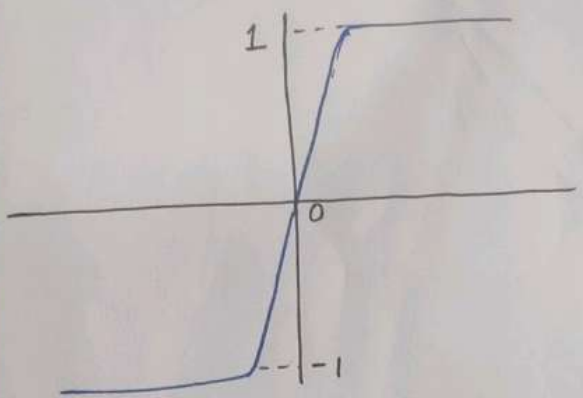
③ Tan h AF

tan (h) = -1 to +1

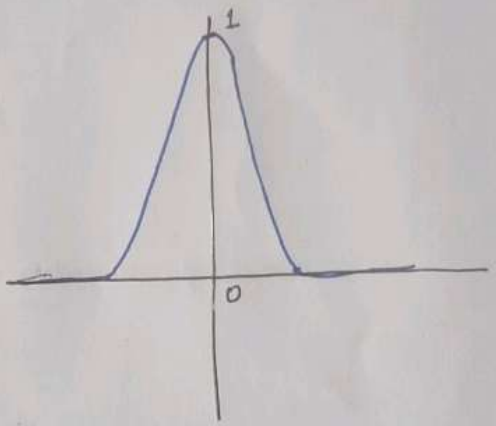
Derivative of tan (h) = 0 to 1

Formula :- $\frac{e^x - e^{-x}}{e^x + e^{-x}}$

Graph of tan h.



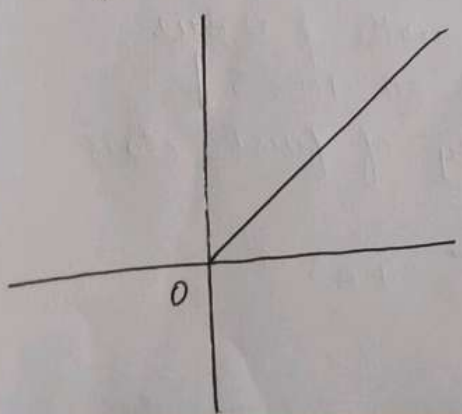
Graph of derivative of tan h



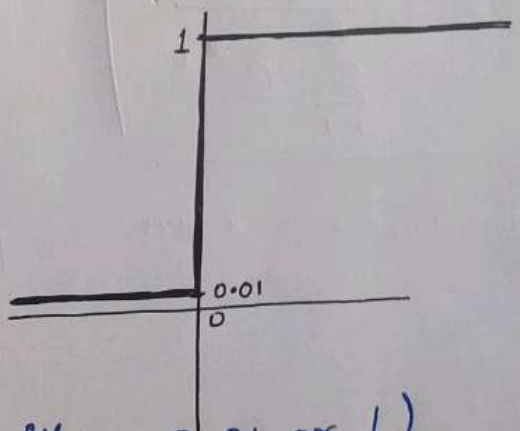
④ Leaky ReLu AF

Formula :- max (0.01x, x)

Graph of leaky ReLu



Graph of derivative of ReLu



(either 0.01 or 1)

⑤ Swish AF

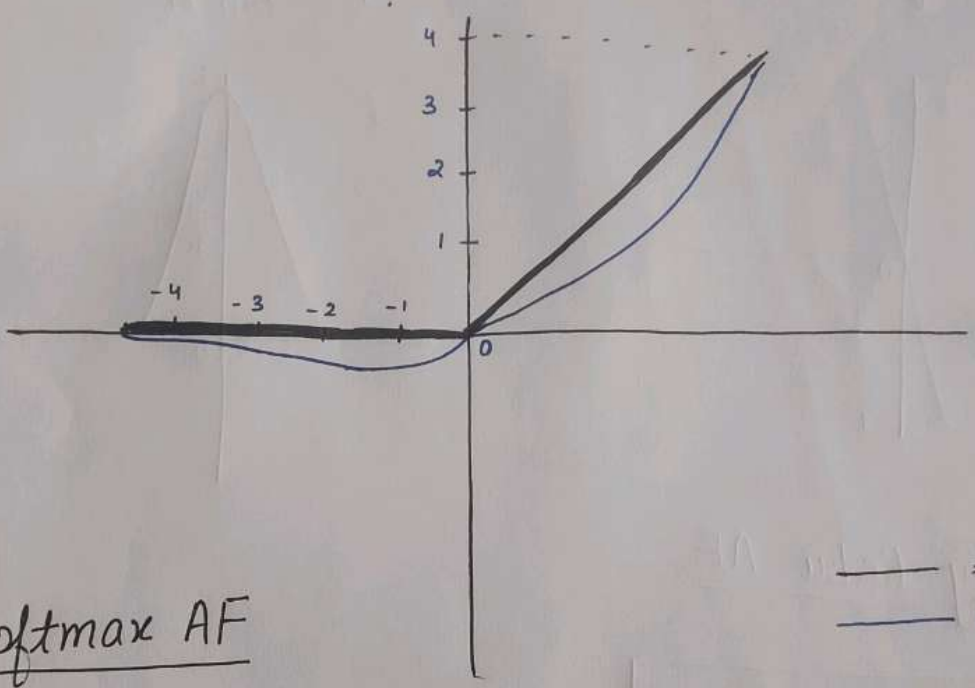
Formula :- $x * \text{sigmoid}(x)$

Here,
$x = w_1x_1 + w_2x_2 + \dots + \text{bias}$.

* Mostly used in LSTM
* Also called self gating
* Use when you have more than 40 layers in your neural network

  If less than 40 layer, it will not work.
* Solves Dead ReLu function.

Graph of ReLu vs Swish.



——— ⇒ ReLu
——— ⇒ Swish.

⑥ Softmax AF

Formula :- $\dfrac{e^{x_j}}{\sum_{K=1}^{K} e^{x_K}}$

$x = w_1x_1 + w_2x_2 + \dots + \text{bias}$.

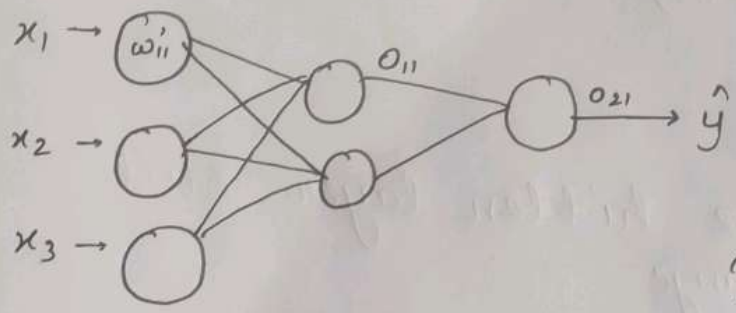
* Used when we have multiple output layers.
* Suppose there are 4 output layers with x value
  $[40, 30, 10, 5]$

finding probability of first class

$$\Rightarrow \frac{e^{40}}{e^{40} + e^{30} + e^{10} + e^{5}}$$

Vanishing Gradient Problem

Previously before invention of ReLu activation fn This was a major problem in sigmoid fn.



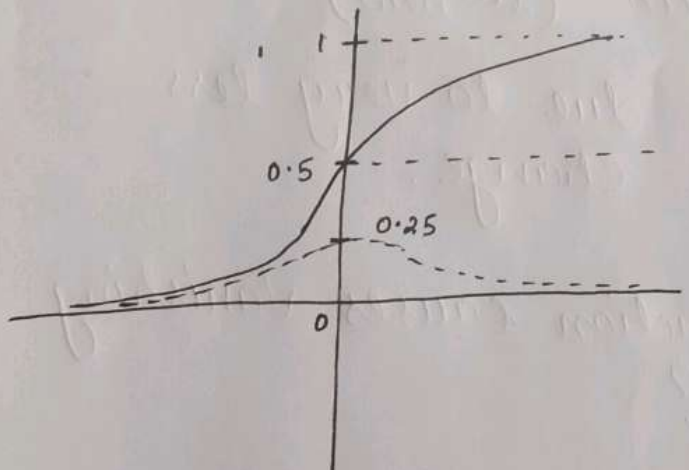
$$\omega'_{11}{}_{new} = \omega'_{11} - \eta \frac{dL}{d\omega'_{11}}$$

$$\therefore \frac{dL}{d\omega'_{11}} = \frac{dO_{21}}{dO_{11}} \cdot \frac{dO_{11}}{d\omega'_{11}}$$

But Main point is :-

Imp ⇒ Derivative of sigmoid fn ranges from 0 to 0.25

- Graph



Here.

—— ⇒ Sigmoid fn

------ ⇒ Derivative of sigmoid fn.

eg. $\frac{dL}{dw_{11}'} = \frac{dO_{21}}{dO_{11}} \times \frac{dO_{11}}{dw_{11}'}$

Suppose their derivative are ⇒ [0.20] [0.02]

⇒ [0.04]

Suppose

$w_{11}^{old} = 2.5$.

$\eta = 1$

So,

$w_{11}^{new}{}' = 2.5 - 1(0.04)$

$= 2.46$.

So,

Here we only had one hidden layer still there is very less change.

If we had many layers, derivative keeps on decreasing by each layer and we would barely get our (w_{new}) as $[w_{new} \approx w_{old}]$ due to very less change.

Q. Which activation function causes vanishing gradient problems?

Ans. ① Sigmoid AF

② Tan h AF

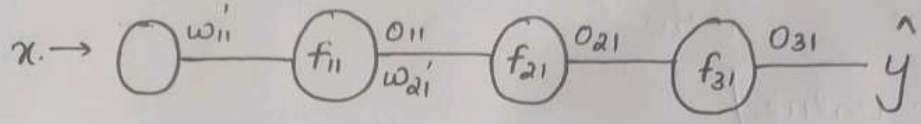
(Its derivative ranges from 0 to 1)

Q. Solution of this problem?

Ans Use of other activation fn like ReLu.

Exploding Gradient Problem

Suppose there is a simple neural network.



$$\omega_{11}'_{new} = \omega_{11}'_{old} - \eta \frac{dL}{d\omega_{11}'}$$

$$\frac{dL}{d\omega_{11}'} = \frac{do_{31}}{do_{21}} \times \frac{do_{21}}{do_{11}} \times \frac{do_{11}}{d\omega_{11}'}$$

{ Considering only one Part of derivative for explanation }

Here $\boxed{z = \omega_{21} \cdot o_{11} + b_2}$

Suppose here activation fn is sigmoid.

We know $\frac{d\phi(z)}{dz}$ ⇒ ranges from 0 to 0.25

$$\Rightarrow \frac{do_{21}}{do_{11}} = \frac{d\phi(z)}{dz} \times \frac{dz}{do_{11}}$$

$$\downarrow$$

$$[0.25] \times \frac{d(\omega_{21} o_{11} + b_2)}{do_{11}}$$

$$\Rightarrow [0.25] \times \omega_{21} \text{ (constant)}$$

Suppose $\omega_{21} = 500$

$$\Rightarrow [0.25] \times 500$$

$$\Rightarrow \underline{125}.$$

We got 125. which is a high value.

Now.

$$\frac{dL}{d\omega_{11}'} = \frac{do_{31}}{do_{21}} \times \frac{do_{21}}{do_{11}} \times \frac{do_{11}}{d\omega_{11}'}$$

$$= 200 \times 125 \times 100$$

$$= \underline{\text{Very high value}}$$

Now, suppose $\eta = 1$

$$w''_{new} = w''_{old} - \eta \frac{dL}{dw''}$$

w''_{old} — less value

$\eta \frac{dL}{dw''}$ — very high value

$$\boxed{w''_{new} = -ve \text{ value}}$$

We can say our old and new weight will vary too much from each other
So will never come close to global minima

Q. What is cause of exploding gradient Problem?

Ans. Weights.

Q. How to solve this problem?

Ans. Proper weight initilization should be done.

Weight Initialization (3 Techniques mentioned.)

- Key Points.

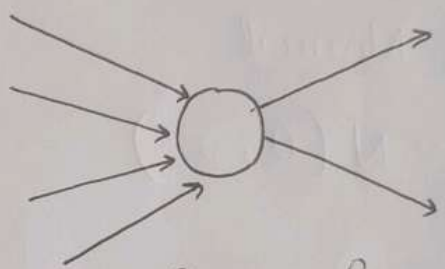
① Weights should be small (Too small → Vanishing Gradient; Too big → Exploding Gradient)

② Weights should not be same (Otherwise output will also remain same from each neuron)

③ Weights should have good variance.

1. Uniform Distribution

$$W_{ij} \sim \text{Uniform}\left[\frac{a}{\sqrt{f_{in}}}, \frac{b}{\sqrt{f_{in}}}\right] = \left[\frac{-1}{\sqrt{f_{in}}}, \frac{1}{\sqrt{f_{in}}}\right]$$



fan in (4) fan out (2)

Here,

W_{ij} → weights.

a → lower limit, b → upper limit } weights are selected within this limit

2. Xavier / Gorat Distribution

- Xavier / Gorat Normal
- Xavier / Gorat Uniform.

Xavier Normal

$$w_{ij} \sim \text{Normal}(0, \sigma)$$

Here,

$$\sigma = \sqrt{\frac{2}{(f_{in} + f_{out})}}$$

Xavier Uniform

$$w_{ij} \sim \text{Uniform}\left[\underbrace{\frac{-\sqrt{6}}{\sqrt{f_{in}+f_{out}}}}_{a}, \underbrace{\frac{\sqrt{6}}{\sqrt{f_{in}+f_{out}}}}_{b}\right]$$

3. He init Distribution

He Normal

$$w_{ij} \sim N(0, \sigma)$$

Here,

$$\sigma = \sqrt{\frac{2}{f_{in}}}$$

He Uniform

$$w_{ij} \sim U\left[\underbrace{-\sqrt{\frac{6}{f_{in}}}}_{a}, \underbrace{\sqrt{\frac{6}{f_{in}}}}_{b}\right]$$

Imp points to Remember :-

① No need to learn formulas by heart. Get basic idea.

② Uniform Distribution, Xavier/Gorat Distribution } ⇒ These workes best with Sigmoid AF.

He init Distribution } ⇒ It works best with ReLu activation fn.

Drop Out & Regularization

Why need this?

If we have a very deep neural network, we will have many weights and biases which will lead to Overfitting.

Fun Fact :- For a deep neural network underfitting never happens.

Two ways to solve Overfitting :-

(i) Regularization (L1, L2)

(ii) Drop Out Layer

Fun Fact :- Developed in around 2013-2014 by Nitish Shrivastav and Jeffry Hinton

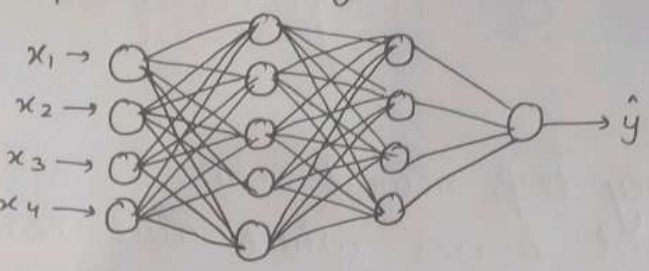
⇒ Nitish was student of Jeffry Hinton.

(1) Drop Out Layer

+ Its principle is based on ML algorith Random forest where full grow DT are made but subset of features is given to that DT. which avoids overfitting problem

Practical exploration

1. Suppose this is your neural network



Suppose this model is overfitting.

2. To create a drop out layer, firstly select a drop-out rate.

It is represented by p.

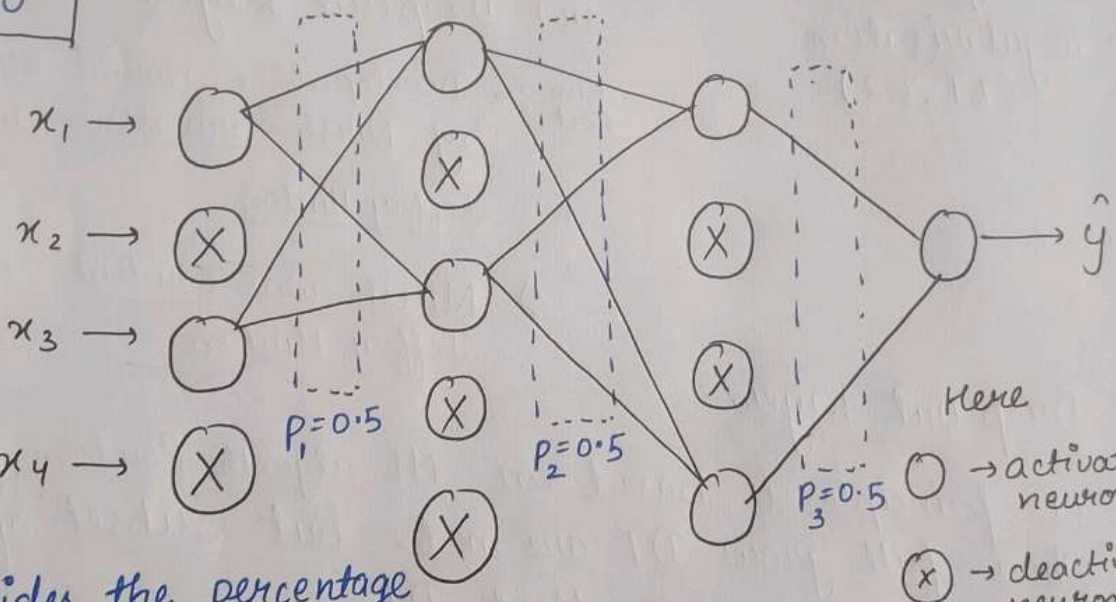
Its range is 0 to 1

$$0 \leq p \leq 1$$

Drop out layer do selection of features randomly

Q. How to find best p-value

Ans. By Hyper Parameter Tuning.



'p' decides the percentage of neuron that gets deactivated

Q. How will this do prediction for test data.

Ans. ① All neuron will get connected as same as in step 1 at this page. with no deactivated neuron

② Now multiply weights with given 'p' value for each layer

eg. $w_1 = w_{1\,old} \times p\text{-value}$.

Optimizers - 1

① Gradient Descent

Formula :- $\omega_{new} = \omega_{old} - \eta \frac{dL}{d\omega_{old}}$

Graph

Imp Terms To Know :-

(i) Epoch

In epoch, whole data is passed at once for every epoch

eg Suppose a data of 1000 record is there

eg. Gradient Descent

(ii) Iteration

In iteration, data is passed one by one.

It is a subunit of epoch

eg. Stochastic Gradient Descent (SGD)

* Gradient Descent uses Epoch approach.

Advantages

① Fast as all data is passed at once

② Only good & fast for small or medium sized dataset

Disadvantages

① Not good for big datasets as, :- large RAM reqd, computationally expensive, More time consuming

② Stochastic Gradient Descent (SGD)

Note: Formula, ~~Graph~~ and working is same as Gradient Descent. Only difference is approach.

SGD uses Iteration approach.

* It means under each epoch certain iteration are given so that only a subset of data is passed to during backpropagation.

eg. For a dataset (500 record), epoch = 10 and iteration = 500

So,

1st epoch → iteration = 500
2nd epoch → iteration = 500
3rd epoch → iteration = 500
⋮
10th epoch → iteration = 500

⇒ Total iteration = 10 × 500 = 5000

Advantages

① It fixes most of disadvantages of Gradient descent like
:- it doesn't require large ram
:- It is not computationally expensive
:- Better than Gradient Descent for big dataset

Disadvantages

① It is time-consuming

as data is passing one by one which takes too much time.

To fix this issue, Minibatch SGD was introduced.

Optimizers - 2

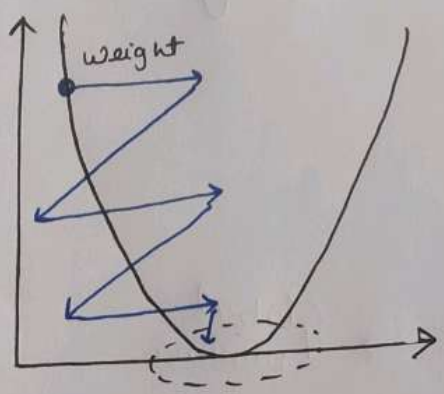
③ Minibatch SGD

It also follow a Iteration approach
For every epoch, a batch size is set.

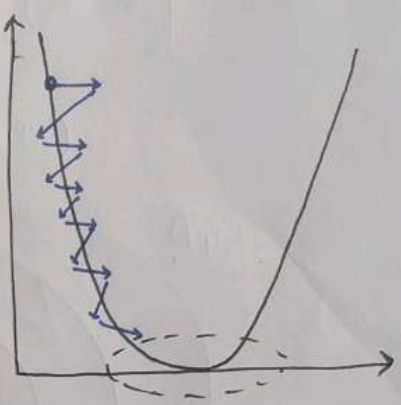
eg. We have 10k records in dataset
epoch = 20
batch size = 1000
So, iteration = $\frac{\text{data}}{\text{batch size}} = \frac{10000}{1000} = 10$.

Now
For every epoch → 10 iteration will be done

Graph of SGD.



Graph of Minibatch SGD



Note: This zig-zag line represent noise.
It is because only a subset of data is provided for weight updation.

Minibatch SGD has less than SGD.
noise

Summary

GD.

Epoch Approch

Full data

eg. Data = 10k records
Epoch = 10

1st epoch → 1 iteration
2nd epoch → 1 iteration
≡
10th epoch → 1 iteration

Note.
1 iteration mean all data is passed at once.

SGD.

Epoch - Iteration approch

Data one by one

eg. Data = 10k records
Epoch = 10

1st epoch → 10k iteration
2nd epoch → 10k iteration
≡
10th epoch → 10k iteration

Note.
iteration = No of record in dataset

Each record is pass one by one.

Mini Batch SGD.

Epoch - Iteration approch

Subset of data

eg. Data = 10k records
Epoch = 10

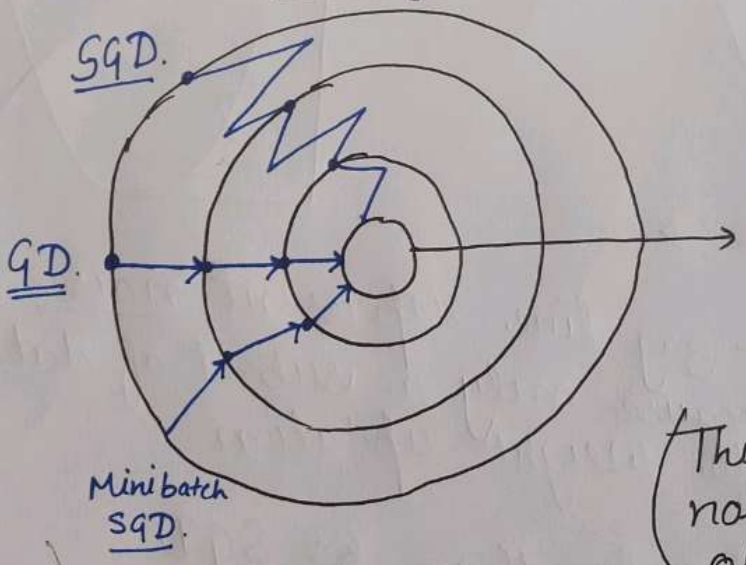
Set a batch size
let it be = 1000

iteration = $\frac{10k}{1000} = \frac{10}{}$

1st epoch → 10 iteration
2nd epoch → 10 iteration
≡
10th epoch → 10 iteration

Note: A subset of 1000 record is passed every iteration.

Convergence



(This describes the noise by each optimizer.)

∴ Noise is due to passing of data one by one or in subset

Optimizers - 3

④ AdaGrad (Adaptive Gradient Descent)

formula is little bit changed here.

Formula :- $w_t = w_{t-1} - \eta'_t \frac{dL}{dw_{t-1}}$

Here,
t = epoch
$t-1$ = previous epoch.

Main idea of this Optimizer :- Different learning rate (η) for different iteration

⇒ Till now, η remains to be fixed but now it will be changing for each iteration for each neuron.

Q. Why need of this adaptive learning rate?

Ans. We know there are two type of features

(i) Dense feature ⇒ most elements are non-zero
(ii) Sparse feature ⇒ most element are zero.

To treat dense & sparse feature effectively we use Adagrad optimizer which takes different learning rate depending upon feature.

eg. Commonly used in Natural Language Processing (NLP) techniques.
like BOW (Bag of Words) { These convert text to vector form }
TF-IDF

Q. How η_t' is determined for each epoch?

Ans. Firstly we set a constant value, generally 0.01

let $\eta = 0.01$

$$\Rightarrow \eta_t' = \frac{\eta}{\sqrt{\alpha_t + \varepsilon}}$$

Here.

ε (epsilon) ⇒ It is generally a small positive number.

$$\alpha_t = \sum_{i=1}^{t} \left(\frac{dL}{dw_i}\right)^2$$

∴ As $\alpha_t \uparrow$, $\eta_t' \downarrow$

It means,

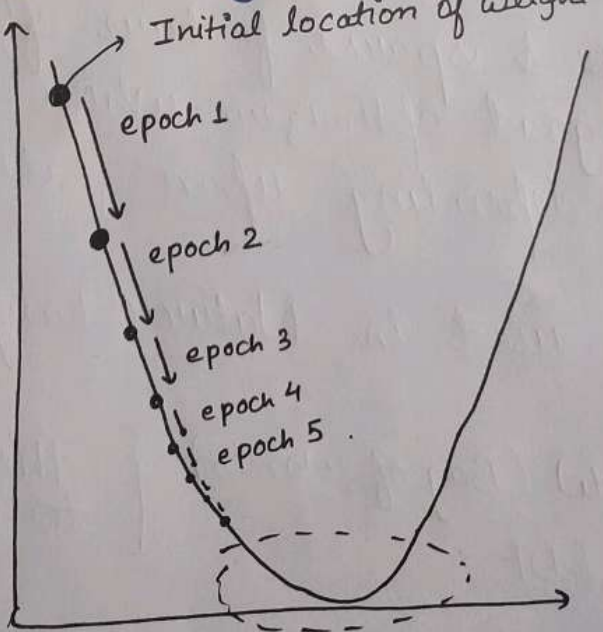
As we proceed further iteration learning rate keep on decreasing.

So, w_t also decreases. fastly at first but as learning rate decreases, w_t also decreases slowly slowly.

Thus it gives a prefect approach to global minima.

let's understand by example.

eg.



As with each epoch, η decreases. Speed of weight updation also decreases.

Optimizer - 4

⑤ AdaDelta

There is one disadvantage of Adagrad that lead to Introduction of Adadelta and RMSPROP Internal working of both is same with minor changes.

Disadvantage :-

$$\eta_t' = \frac{\eta}{\sqrt{\alpha_t + \varepsilon}}$$

$$\alpha_t = \sum_{i=1}^{t} \left(\frac{dL}{dw_i}\right)^2$$

Sometimes, α_t become very very big as we are squaring the derivative.

If $\alpha_t \uparrow\uparrow$ Then $\eta_t' \downarrow\downarrow$

Due to this it takes very long to reach global minima.

as $w_{new} \simeq w_{old}$ or

$w_t \simeq w_{t-1}$

Solution to this Disadvantage :-

We have to restrict α_t from becoming too large.

This is what AdaDelta and RMSPROP do.

There is change in formula.

Formula $\Rightarrow$ $\underset{\text{new}}{w_t} = \underset{\text{old}}{w_{t-1}} - \eta'_t \frac{dL}{dw_{t-1}}$

Change is, Firstly take $\eta = 0.01$

Then.

$$\eta'_t = \frac{\eta}{\sqrt{W_{avg} + \varepsilon}}$$

Here.
$\varepsilon \Rightarrow$ Small +ve value.

$W_{avg} \Rightarrow$ Weighted average.

Now.

$$W_{avg_t} = \gamma \times W_{avg_{(t-1)}} + (1-\gamma) \times \left(\frac{dL}{dw_t}\right)^2$$

Here,

$\gamma \Rightarrow$ Generaly $\underline{0.95}$

Q. Why replace α_t with W_{avg}?

Ans. Look at both formulat.

$$\alpha_t = \sum_{i=1}^{t} \left(\frac{dL}{dw_t}\right)^2$$

$$W_{avg_t} = \gamma \times W_{avg_{(t-1)}} + (\gamma - 1) \times \left(\frac{dL}{dw_t}\right)^2.$$

Reason 1:- There is no summation in W_{avg} formula.

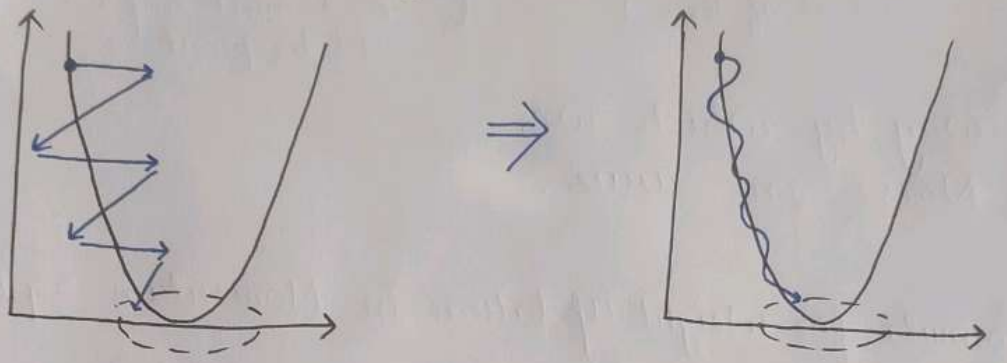
Reason 2:- Even if $\left(\frac{dL}{dw_t}\right)^2$ is a very big number but it is multiplied to $(1-\gamma)$ ie a very small value

$(1-\gamma) = 1 - 0.95 = \underline{0.05}$

Optimizers - 5

⑥ SGD with Momentum

Main aim of this optimizer :- To reduce the noise (zig zag line)
To reduce time for convergence to global minima



* It smoothens the zig zag line.

How it happens? ⇒ By concept of Exponentially Moving Average

⇒ Suppose we have different data points (represented by 'b') at different time intervals (represented by 't')

$t_1 \quad t_2 \quad t_3 \quad \text{-------} \quad t_n$
$b_1 \quad b_2 \quad b_3 \quad \text{-------} \quad b_n$.

⇒ Formula will be :-

Here. $\gamma \Rightarrow 0$ to 1
generally 0.5 or 0.9

$V_{t_1} = b_1$

$V_{t_2} = \gamma \times V_{t_1} + b_2 \Rightarrow 0.5\, b_1 + b_2 \quad -①$

$V_{t_3} = \gamma \times V_{t_2} + b_3 \Rightarrow \gamma^2 \times b_1 + \gamma \times b_2 + b_3$

$= \gamma(\gamma \times V_{t_1} + b_2) + b_3 \Rightarrow 0.25\, b_1 + 0.5\, b_2 + b_3 \quad -②$

Main thing to Understand :- Look at eq ①.

$0.5b_1 + b_2$

Constant of $b_1 = 0.5$
Constant of $b_2 = 1$.

It means b_2 point was given more importance than b_1

Now look at eq ②

$0.25b_1 + 0.5b_2 + b_3$

Constant of $b_1 = 0.25$
Constant of $b_2 = 0.5$
Constant of $b_3 = 1$

It means least importance given to b_1 point.
Then more importance to b_2 point
And most importance to b_3 point.

⇒ This is way by which we reduce Noise in curve.

Not much important ⇒ Formula for Weight Updation in Momentum SGD.

$$\omega_{new} = \omega_{old} - \left[\gamma V_{t-1} + \eta \frac{dL}{d\omega_{old}} \right]$$

Here.

$$V_{t-1} = 1 \times \left[\frac{dL}{d\omega_{old}}\right]_t + \gamma \left[\frac{dL}{d\omega_{old}}\right]_{t-1} + \gamma^2 \left[\frac{dL}{d\omega_{old}}\right]_{t-2} - - - -$$

Q. Why word Momentum is there?

Ans. In above formula,
The term

γV_{t-1} ⇒ It is called the momentum.

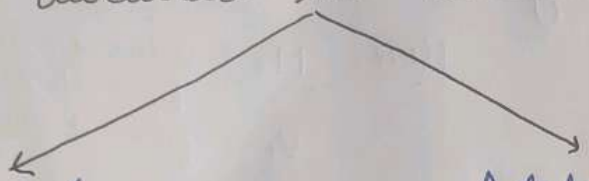
Note: No need to remember complex equation and formulas. Just get a basic idea of it.

Optimizer - 6

⑦ Adam (Adaptive Moment estimation)

⇒ Most used optimizer in Neural networks.

It is the combination of two powerful techniques that we discussed in earlier optimizers.



Momentum
(Taken from SGD Momentum)

Adaptive learning Rate
(Taken from Adagrad / Adadelta)

* Formula

$$w_{new} = w_{old} - \frac{\eta}{\sqrt{V_t + \varepsilon}} \times M_t$$

Here,

η ⇒ learning rate (generally 0.01)

ε ⇒ small +ve value.

V_t ⇒ represent from Adagrad for adaptive learning rate.

M_t ⇒ represents for Momentum.

V_t =

Now,

$$M_t = \beta_1 M_{t-1} + (1-\beta_1)\omega_t \quad \rightarrow \text{Momentum.}$$

$$V_t = \beta_2 V_{t-1} + (1-\beta_2)(\omega_t)^2 \quad \rightarrow \text{Adagrad}$$

Then there is something called <u>Bias Correction</u>

$$\hat{M}_t = \frac{M_t}{1-\beta_1^t}$$

$$\hat{V}_t = \frac{V_t}{1-\beta_2^t}$$

Here, M_t → value of momentum from above

V_t → from above.

t → represent epoch number (don't misguide it with squaring)

[β_1 → generally 0.9
β_2 → generally 0.99]

↳ as by keras library.

Now put these $\hat{M}_t$ and $\hat{V}_t$ value in place of M_t and V_t value in main formula at previous page to get new weight.

Loss Functions - 1

Q. What is loss function?

Ans. It is a method of evaluating how well your model is performing with dataset.

High loss ⇒ Poor model
Less loss ⇒ Great model

Most basic loss function :- $(y - \hat{y})^2$

where y → actual value
$\hat{y}$ → value predict by model.

Types of Loss Function

and many more.....

- **Object Detection**
  - → Focal loss.
- **Regression**
  - → MAE
  - → MSE
  - → Huber loss.
- **Classification**
  - → Binary crossentropy
  - → Categorical crossentropy
- **Autoencoders**
  - → KL divergence

Q. Difference between Loss Function and Cost Function?

Ans: Loss function is single training unit for single data record
Cost function is summation of all loss function.

eg. Loss function
⇒ $(y - \hat{y})^2$

Cost function
⇒ $\frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$

① Mean Squared Error (MSE)

Used in Regression problems.

Here, y_i → actual value

$\hat{y}_i$ → predicted value

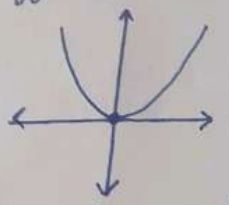
Formula :- $(y_i - \hat{y}_i)^2$

Q. Why squaring in formula?

Ans. Because if value comes out to be negative. With squaring we convert it to positive.

Advantages

1. Easy to understand.
2. Differentiable Curve.



It helps us during optimization process.

3. 1 local minima.

∴ Some loss fn have multiple minimas which gives us different solution during backpropagation.

Disadvantages

1. Unit is not same as we have done squaring.

   If if have actual unit as cm unit in loss fn will $(cm)^2$

2. Not robust to outliers.

Loss Functions - 2

② Mean Absolute Error (MAE)

Used for Regression problems

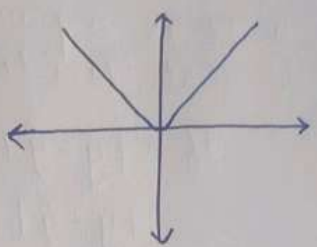
Formula :- $|y_i - \hat{y}_i|$

Advantages

1. Easy to understand
2. Unit is same.
3. Robust to outliers as compared to MSE

Disadvantages

1. Not differentiable which leads to problem during optimization.



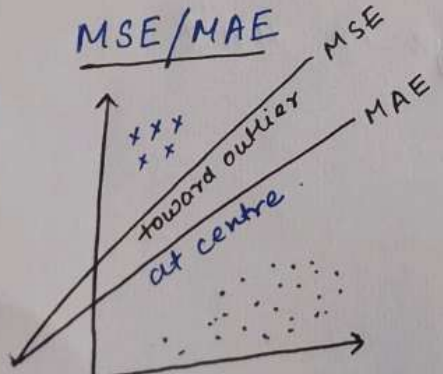
③ Huber Loss

Used for Regression problems.

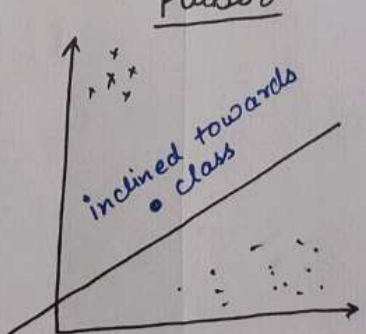
When to use :- When we have a greater amount of outlier or when we want the best regression line in a imbalance data.

eg. Data ⇒ 75% data is •
25% data is *

MSE/MAE



Huber



It gives weightage to high percentage class.

It is combination of MSE and MAE

Formula :- $$L = \begin{cases} \frac{1}{2}(y-\hat{y})^2 & , \text{if } |y-\hat{y}| \leq \delta \\ \delta|y-\hat{y}| - \frac{1}{2}\delta^2 & , \text{if } |y-\hat{y}| > \delta \end{cases}$$

δ → It is a parameter that decides whether a point is outlier or not.

If $|y-\hat{y}| \leq \delta$ → Not outlier

$|y-\hat{y}| > \delta$ → outlier

If a point is outlier, Huber loss acts as MAE

a point is not outlier, Huber loss acts as MSE.

Advantages

1. Fully Robust to outliers
2. Differentiable.

Disadvantages

1. Need hyperparameter tuning to find best value for δ.
2. Not used much as compared to MSE and MAE.
3. Change in unit

Loss Functions - 3

④ Binary Cross Entropy

Used for Classification Problem

* Can only be used for Two class classification
  eg. Cat or Dog
  Pass or Fail

Formula :- $L = -y \log(\hat{y}) - (1-y) \log(1-\hat{y})$

Here, y → actual value.
$\hat{y}$ → predicted value

Fun Fact :- It is mostly used in Logistic Regression in Machine learning.

Advantages

1. Differentiable
2. Converges faster.
3. Best for two class classification

Disadvantages

1. It has multiple local minima.
2. Not easy to understand the internal working as compared to MSE, MAE, etc.
3. Sensitive to class imbalance.
   eg. Data has 100 records
   A class → 80 records
   B class → 20 records.

   Model become biased towards A class.

⑤ Categorical CrossEntropy

Used for classification function.

⇒ Used for multi-class classification

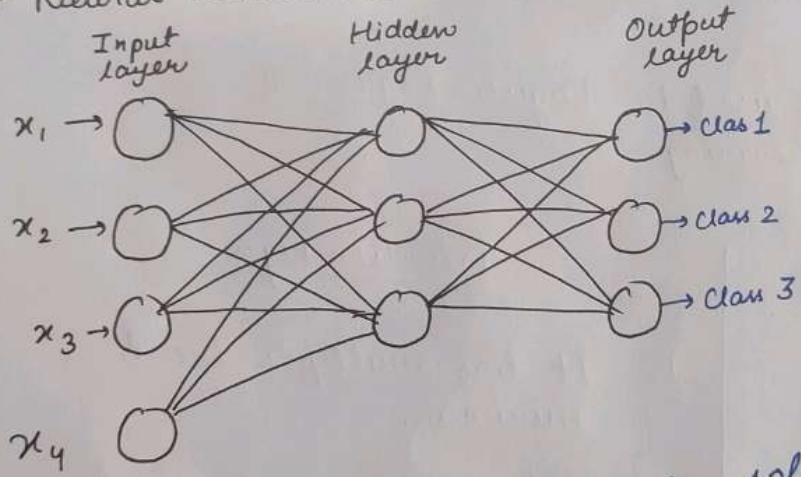
Formula :- $$L = -\sum_{j=1}^{K} y_j \log(\hat{y}_j)$$

Here
K → no. of classes

Suppose we have 3 classes in our data
So formula will be,

$$L = -y_1 \log(\hat{y}_1) - y_2 \log(\hat{y}_2) - y_3 \log(\hat{y}_3)$$

* Neural Network looks like :-



Here,
In multiclass classification,
No of neuron in output layer = No of classes in dataset

Note :- Important :- Use only softmax activation fn in output layer of a multi-class classification problem.